P2663R6
Interleaved complex values support in std::simd

Published Proposal, 2025-01-13

This version:
http://wg21.link/P2663R6

Authors:
Daniel Towner (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The paper proposes to support interleaved complex values in `std::simd`

Table of Contents

1. Motivation

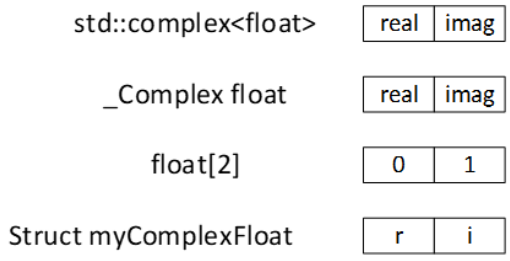
ISO/IEC 19570:2018 (1) introduced data-parallel types to the C++ Extensions for Parallelism TS. [P1928R15] is proposing to make that a part of C++ IS. Intel supports the concept of a standard interface to SIMD instruction capabilities and we have made extra suggestions for other APIs and facilities for `std::simd` in document [P2638R0]. One of the extra features we suggested was the ability to work with interleaved complex values. It is now becoming common for processor instruction sets to include native support for these operations (e.g., Intel AVX512-FP16, ARM Neon) and we think that allowing `std::simd` to directly access these features is advantageous.

This document gives more details about the motivation for supporting complex-values, the different storage and implementation alternatives, and some suggestions for a suitable API.

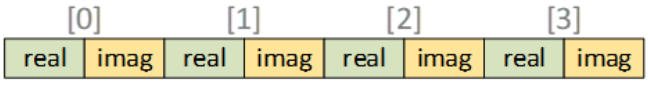
§ 2. Background

Complex-valued mathematics is widely used in scientific computing and many types of engineering, with applications including physical simulations, wireless signal processing, media processing, and much more besides. C++ already supports complex-valued operations using the `std::complex` template available in the `<complex>` header file and C supports complex values by using the `_Complex` keyword. C++ supports floating point elements, including `std::float16` (C++23), `float`, `double` or `long double`, though in practice compilers permit integer complex values too. Such is the importance of interleaved complex values that some mainstream processor vendors now provide native hardware support for complex-value SIMD instructions (e.g., Intel AVX512-FP16, ARM Neon v8.3).

Any complex value is represented as a pair of values, one for the real component and one for the imaginary component. In C and C++ a complex value is a pair of two values of the appropriate data type, allocated to a single unit containing 2 elements which are stored contiguously. This is illustrated in the figure below. This storage layout is used in other languages too, allowing for easy data interchange between software written in different languages, or with comparable user-defined types. This format may also be used for hardware accelerators or interfaces too:



When many complex values are stored together, such as in a C-style array, a C++ `std::array` or a C++ `std::vector`, then the real and imaginary elements are interleaved in memory:



Many applications, libraries, programming languages and interfaces between diverse software and hardware components will store data in this interleaved format, and it is treated as the default layout for blocks of complex-valued data. To improve the compute efficiency of this data format both Intel and ARM have introduced native hardware support to allow efficient SIMD operations to be performed in this data format without having to resort to reformatting the data into a more SIMD-amenable form. Where the underlying target hardware does not have native support for interleaved complex values it is straight-forward to synthesize a sequence of operations which give the same effect.

Given the popularity of interleaved complex-values as a data exchange and compute format we propose that `std::simd` should be able to directly represent this simd format in a form exemplified as:

```
std::simd<std::complex<float>, 5>
std::simd<std::complex<double>, 9>
```

In all these cases the fundamental element type will be a suitable complex value, and the individual components of each complex element will be worked on as a single unit, according to the rules of complex arithmetic where appropriate. This includes:

- All numeric operators (e.g., `+`, `/`, `-`, `*`) and their derivatives (e.g., assignment operators) will operate as per their standard complex type definition. For example, the multiply operator will invoke a complex multiply for each complex element.
- Any numeric operation will apply the operation on a per-element basis, generating a simd object of the same size as the input, but modifying the element type to be the appropriate return type. For example, when applied to a `simd<complex<T>>`:
  - `exp`, `sin`, `log`, `tan`, `cos` and so on will generate a `simd<complex<T>>`
  - `abs` or `norm` will generate a `simd<T>` (i.e., real-valued SIMD equivalent).

  Note that the mathematical behavior will also be applied appropriately (e.g., if a real or imaginary component is a NaN, then the complete operation for a complex operation will also be NaN).
- The `simd_mask` associated with a `simd<complex<T>>` will have each individual mask bit refer to a complete complex element, not to sub-components of the complex elements. When the `simd_mask` is backed by a compact representation (e.g., AVX-512), then each individual bit will refer to a complete complex element. When the `simd_mask` is backed by an element mask (i.e., multiple bits filling a container of the same size as the simd element) then the size of each bit element will be twice the size of the underlying complex value type (e.g., a `simd_mask<complex<_Float16>>` would be equivalent to something like `simd<uint32_t>`, since `sizeof(uint32_t) == 2 * sizeof(_Float16)`.
- Operations or functions which are invalid for complex values will be removed. This includes relational operators (e.g., `<`, `<=`, `>`, `>=`) and those derived from them such, as `min` or `max`.
- Any operation which moves or reorders elements will move entire complex values. For example, any sort of permute, resize, split, concat, insert, extract or indexing operation works on complete complex elements as though they are atomic units.

- Reduction operations apply at the level of complex values (this follows from the previous bullet), provided the mathematical operation is valid (e.g., reduce-max would not be available).

One of the aims of `std::simd` is to make it possible to write a generic code, which can use either a scalar type or a data-parallel type interchangeably. To this end, `std::simd` provides overloads of many of the standard functions which operate in data-parallel mode. The same should apply to a simd of complex values; it should be possible to write an algorithm which uses either `std::complex<T>` or `simd<std::complex<T>>` with only a type replacement, not with major API changes. To this end `std::simd` should include some member functions, such as real or imag, to mirror those found in std::complex.

Note that an alternative way to create parallel complex values is to allow `std::complex` to use simd elements, such as `std::complex<simd<float>>`. This storage layout is called separated storage and discussion of this format is outside the scope of this paper.

§ 3. Overview of updates required to support `std::complex` simd elements

There are three ways in which `std::simd` needs to change to accommodate `std::complex` elements:

- Modify the definition of vectorizable types and the operations on them to allow for complex elements. In common with `std::complex` itself, only specializations of floating-point element types are allowed. These modifications are small in nature and don't change the fundamental `std::simd` API.
- Add a small number of methods to `std::simd` to mimic those found in `std::complex`. These modifications allow a `std:::simd<complex<T>>` value to be easily inserted into source code by text or template replacement in place of a `std::complex` value without requiring a rewrite.
- Add a set of overloads and free functions to allow `simd<complex<T>>` to reflect the different behaviors of complex (e.g., `sin`, `cos`, `exp`, `log`).

Each of these points will now be discussed in more detail.

§ 3.1. `std::simd` base modifications

In its current definition, `std::simd` allows the element type to be any cv-unqualified arithmetic type other than `bool` or `long double`. The first modification is to extend the set of allowable element types to include any `std::complex<>` specialisation for a cv-unqualified vectorizable floating-point type.

Any simd operation or function which relies only on an element being an atomic container which can be arbitrarily moved or duplicated as a single unit will continue to operate as expected. For example, all the following will work on `simd<complex<T>>` without modification:

- Constructors or `copy_to`/`copy_from` functions for loading and storing values to and from contiguous iterators
- Broadcast constructor
- Index operations
- Reordering operations, such as `simd_split`, `simd_cat`, `permute`, `resize`, `insert` and `extract`.
- Masking selection operations (using a mask to include or exclude values from an operation or copy).

`simd_mask`'s of complex values do not need to have any special behaviour since the mask is simply a collection of boolean values, and the underlying element type makes no difference.

Constructors, including generator functions, which work on atomic units (e.g., broadcasting constructor) will work without modification. Note that the existing broadcast constructor will already permit a real-valued object to be used in a broadcast, since an individual value may be converted to a complex value with a zero imaginary:

```
auto x = simd<std::complex<float>>(3.14f);
// [3.14 + 0i, 3.14 + 0i, ...]
```

Numeric operations (binary, unary, and compound-assignment) are mostly covered by [P1928R15] remark:

A simd object initialized with the results of applying the indicated operator to `lhs` and `rhs` as a binary element-wise operation

In the case of complex values this has the practical effect of ensuring that operators will perform their complex operations without changing the definition of `std::simd` (e.g., `operator*` will perform complex-valued multiplication). Furthermore, [P1928R15] notes that constraints will be applied as necessary on operators and functions:

Constraints: Application of the indicated operator to objects of type `T` is well-formed.

This will ensure that operators which are not permitted for complex numbers (e.g., `operator%`, `operator<<`, `operator&`) will be removed from the overload set, as will functions which rely on

these operators (e.g., `min`, `max`, `clamp`).

§ 3.2. Additional complex methods for `std::simd`

It is desirable for `std::simd` to be source- or template-replaceable with respect to its underlying element type. For example, given a simple code fragment which works with `std::complex`:

```
auto my_fn(auto cmplx_value)
{
    std::complex<float> rotator (0, 1);
    return (cmplx_value * rotator).real();
}
```

In this example the function will work with a `std::complex<float>` input. Ideally it should also work with an input of type `simd<complex<float>>`. For this to happen, firstly the `operator*` must work with a scalar value, and secondly it should be possible for the `real()` method to be invoked on a `simd<complex<T>>` value.

`std::complex` provides a number of member functions:

- Constructors
- `operator=` (assignment)
- `operator+=`, `operator-=`, `operator*=`, `operator/=`
- `real`
- `imag`

The assignment operator and the compound-assignment operators are already provided by `std::simd` so no further action is required. The remaining member functions that are required are considered in the following sub-sections.

§ 3.2.1. Constructors

Conversion and copy methods already exist in `std::simd` and can be used for complex SIMD values. An additional constructor is needed to match the `std::complex` constructor which can build

a complex value from separate real and imaginary components: constructed from real and imaginary simd values.

```
template<simd-floating-point VC>
constexpr explicit(see below)
basic_simd(const VC& reals, const VC& imags = {}) noexcept;
```

This constructor is constrained so that it can only be used for SIMD values with complex elements. Like its `std::complex` counterpart it can be used to build a complex value from real components only, with insertion of zero imaginaries. This constructor can be used as in the following example:

```
simd<float, 8> reals = ...;
simd<float, 8> imaginaries = ...;

// Create a real-valued complex number (i.e., zero imaginaries)
simd<std::complex<float>, 8> realAsComplex (reals);

// Create a complex value from real and imaginary simd inputs
simd<std::complex<float>, 8> cmplx (reals, imaginaries);
```

3.2.2. Real/imag accessors

It should be possible to separately read or write the real and imaginary components. For example, given a simd of `std::complex<float>` values, the real components of the simd will be extracted as a value of type `std::simd<float>` instead. Similarly, given a `simd<float>` value those values can be inserted into the simd as the real components of each respective value.

The read accessors for `std::complex` come in two variants: a free function, and a member function. These would be defined in `std::simd` as follows:

```
// Free functions for accessing real/imaginary components
template <class T, class ABI> constexpr auto real(const basic_simd<complex<
template <class T, class ABI> constexpr auto imag(const basic_simd<complex<

// Member functions of std::simd for accessing real/imaginary components.
constexpr simd<T, size()> std::simd<std::complex<T>>::real() const;
constexpr simd<T, size()> std::simd<std::complex<T>>::imag() const;
```

The write accessors for `std::complex` are provided as member functions which allow the real and imaginary components of an existing complex value to be overwritten.

```
// Member functions of std::simd for accessing real/imaginary components.
template<class T, class ABI>
constexpr void real(const basic_simd<T, ABI>& reals);

template<class T, class ABI>
constexpr void imag(const basic_simd<T, ABI>& imags);
```

The advantage of making `real` and `imag` members of `std::simd` is an API consistency between `std::complex<T>` and `std::simd<std::complex<T>>` for both getters and setters.

`std::complex` has a variety of overloads for these accessors for special cases. For example:

```
template<class FloatingPoint> constexpr FloatingPoint real(FloatingPoint f)
template<class Integer>       constexpr double        real(Integer i);

template<class FloatingPoint> constexpr FloatingPoint imag(FloatingPoint f)
template<class Integer>       constexpr double        imag(Integer i);
```

In some cases, mirroring these functions into `simd<complex<T>>` would result in an unwanted performance degradation as, for example, the potentially small `simd<Integer>` would be turned into a much larger `simd<double>` result, which is likely not what the user intended. For `simd<complex<T>>` we therefore propose that only sufficient overloads are provided to operate on `simd<complex<T>>` values.

§ 3.3. Additional free functions

The free functions for `std::complex` include numeric operators, such as `+`, `-`, `*` and `/`. These are already proposed in [P1928R15] and need not be considered further.

The remaining free functions which should be specialized for `std::simd<complex<T>>` include:

`real(simd)`	returns the real part
`imag(simd)`	returns the imaginary part

`abs(simd)`	returns the magnitude of a complex number
`arg(simd)`	returns the phase angle
`norm(simd)`	returns the squared magnitude
`conj(simd)`	returns the complex conjugate
`proj(simd)`	returns the projection onto the Riemann sphere
`polar(simd)`	constructs a complex number from magnitude and phase angle
`exp(simd)`	complex base e exponential
`log(simd)`	complex natural logarithm with the branch cuts along the negative real axis
`log10(simd)`	complex common logarithm with the branch cuts along the negative real axis
`pow(simd)`	complex power, one or both arguments may be a complex number
`sqrt(simd)`	complex square root in the range of the right half-plane
`sin(simd)`	computes sine of a complex number
`cos(simd)`	computes cosine of a complex number
`tan(simd)`	computes tangent of a complex numbe
`asin(simd)`	computes arc sine of a complex number
`acos(simd)`	computes arc cosine of a complex number
`atan(simd)`	computes arc tangent of a complex number
`sinh(simd)`	computes hyperbolic sine of a complex number
`cosh(simd)`	computes hyperbolic cosine of a complex number
`tanh(simd)`	computes hyperbolic tangent of a complex number
`asinh(simd)`	computes area hyperbolic sine of a complex number
`acosh(simd)`	computes area hyperbolic cosine of a complex number
`atanh(simd)`	computes area hyperbolic tangent of a complex number

All of these functions should operate element-wise in the same way as their scalar counterparts. Note that although most of these functions take complex inputs and generate complex outputs, some of these functions generate real-valued outputs (e.g., `abs` returns the magnitude, which is real-valued), or accept an input which is real-valued (e.g., `conj` can accept a real-valued value and return a complex value).

Unlike other math functions in `simd`, none of the `simd` free-functions equivalents of `std::complex` are marked as `noexcept`. This matches the behaviour of `std::complex`.

In `std::complex` a variety of overloads exist for each of the functions listed above, such as with `conj`:

```
template<simd-type V>          constexpr V conj(const V& v);
template<class FloatingPoint> constexpr FloatingPoint conj(FloatingPoint f)
template<class Integer>        constexpr double        conj(Integer i);
```

If these overloads were mirrored into `simd<complex>` then they could result in unexpected performance degradation. For example, accepting a `simd<Integer>` and returning a `simd<double>` will typically result in a much larger SIMD register being required, or allowing an argument to be a slightly different type to what is typical using an overload may result in an implicit conversion with an unwanted cost. It is the intent of `simd<complex>` to support only complex-type operations on `simd<complex<T>>`, not on `simd<Other>`. if the user wants an overload-like behaviour with `simd<complex<T>>` they should handle the appropriate conversion themselves so that the performance expectations can be managed.

§ 4. Implementation experience

Intel have written an example implementation of `std::simd` which includes the extensions to support interleaved complex values. We have been able to generate efficient code both on targets with native support (e.g., AVX512_FP16), and also on targets without native support which require synthesized code sequences.

§ 5. Questions for LEWG from LWG

During LWG review two queries were raised concerning design intent which need to be reconfirmed by LEWG:

- In common with `std::complex` itself, the `simd` equivalents of the `std::complex` functions are not marked as `noexcept`. This design is different to the bulk of `std::simd` where functions are typically marked as `noexcept`.
- Only sufficient overloads for accessors and free-functions to support `simd<complex<T>>`. Overloads for `simd<float>`, `simd<Integer>`, and `simd<Other>` are not provided.

§ 6. Wording

The wording relies on [P1928R15] being landed to the Working Draft. Below, substitute the `&#xFFFD;` character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 6.1. Modify [**simd.general**]

`&#xFFFD;`: The set of vectorizable types comprises all standard integer types, character types, the types `float` and `double`, and `complex<T>`, where `T` is a vectorizable floating-point type . ([basic.fundamental]). In addition, `std::float16_t`, `std::float32_t`, and `std::float64_t` are vectorizable types if defined ([basic.extended.fp]).

§ 6.2. Modify [**simd.expos**]

Add a concept to detect a `simd` type:

```
template<class V>
  concept simd-type = // exposition only
     same_as<V, basic_simd<typename V::value_type, typename V::abi_type>>
     is_default_constructible_v<V>;
```

Add new concepts to detect `simd` types with specific element type requirements:

```
template<class V>
  concept simd-complex = // exposition only
    same_as<typename V::value_type, complex<typename V::value_type::value

template<class V>
  concept simd-integral = // exposition only
    simd-type<V> && integral<typename V::value_type>;
```

Modify existing type-element concepts to use the new refactored `simd-type` concept:

```
template<class V>
  concept simd-floating-point = // exposition only
    same_as<V, basic_simd<typename V::value_type, typename V::abi_type>>
    is_default_constructible_v<V> &&
    simd-type<V> &&
    floating_point<typename V::value_type>;
```

§ 6.3. Modify [**simd.syn**]

In the header `<simd>` synopsis - [simd.syn] - add at the end after the "Mathematical functions"

```
// [simd.complex], basic_simd complex functions
template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> real(cor

template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> imag(cor

template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> abs(cons

template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> arg(cons

template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> norm(cor

template<simd-complex V> constexpr V conj(const V&);
template<simd-complex V> constexpr V proj(const V&);
template<simd-complex V> constexpr V exp(const V& v);
template<simd-complex V> constexpr V log(const V& v);
template<simd-complex V> constexpr V log10(const V& v);

template<simd-complex V> constexpr V sqrt(const V& v);
template<simd-complex V> constexpr V sin(const V& v);
template<simd-complex V> constexpr V asin(const V& v);
template<simd-complex V> constexpr V cos(const V& v);
template<simd-complex V> constexpr V acos(const V& v);
template<simd-complex V> constexpr V tan(const V& v);
template<simd-complex V> constexpr V atan(const V& v);
template<simd-complex V> constexpr V sinh(const V& v);
template<simd-complex V> constexpr V asinh(const V& v);
template<simd-complex V> constexpr V cosh(const V& v);
template<simd-complex V> constexpr V acosh(const V& v);
template<simd-complex V> constexpr V tanh(const V& v);
template<simd-complex V> constexpr V atanh(const V& v);

template<simd-floating-point V>
```

```
  rebind_simd_t<complex<typename V::value_type>, V> polar(const V& x, cor

template<simd-complex V, simd-complex V> constexpr V pow(const V& x, cons
```

6.4. Modify [**simd.overview**]

In the header `<simd>` overview - [simd.overview] - add at the end of the existing list of constructors.

```
template<simd-floating-point VC>
constexpr explicit(see below)
basic_simd(const VC& reals, const VC& imags = {}) noexcept;
```

Add after the operators:

```
// [simd.complex_accessors], basic_simd complex-value accessors
constexpr rebind_simd_t<typename T::value_type, basic_simd> real() const;
constexpr rebind_simd_t<typename T::value_type, basic_simd> imag() const;
template<class CAbi>
constexpr void real(const basic_simd<typename T::value_type, CAbi>& v);
template<class CAbi>
constexpr void imag(const basic_simd<typename T::value_type, CAbi>& v);
```

6.5. Modify [**simd.summary**]

> NOTE: This paragraph was removed from [P1928R12]. Should it be re-inserted to allow the behaviour of complex to be specified?

[illegible]: Default intialization performs no initialization of the elements , except when the value type is a `complex` specialization in which case all elements will be value-initialized ; value-initialization initializes each element with `T()`. [Note: Thus, default initialization leaves non-complex elements in an indeterminate state. — end note]

§ 6.6. Modify [**simd.ctor**]

Provide a `complex`-value constructor for real/imaginary component-wise initialization.

```
template<simd-floating-point VC>
constexpr explicit(see below)
basic_simd(const VC& reals, const VC& imags = {}) noexcept;
```

Constraints:

- *simd-complex*<basic_simd> is `true`.
- *simd-size-v*<VC> == size() is `true`.

Effects:

- Initializes the i^{th} element with `value_type(reals[i], imags[i])` for all `i` in the range of `[0, size())`.

Remarks:

- The expression inside `explicit` evaluates to `false` if and only if the floating-point conversion rank of `T::value_type` is greater than or equal to the floating-point conversion rank of *VC*`::value_type`.

§ 6.7. Add new section [**simd.complex_accessors**]

`simd` complex accessors [simd.complex_accessors]

```
constexpr rebind_simd_t<typename T::value_type, basic_simd> real() const;
constexpr rebind_simd_t<typename T::value_type, basic_simd> imag() const;
```

Constraints:

`simd-complex``<basic_simd>` is `true`.

Returns:

A data-parallel object initialized with values equal to the results of applying *cmplx-func* element-wise to `x`, where *cmplx-func* is either `real` or `imag` corresponding to the overload name.

```
template<class CAbi>
constexpr void real(const basic_simd<typename T::value_type, CAbi>& v);
template<class CAbi>
constexpr void imag(const basic_simd<typename T::value_type, CAbi>& v);
```

Constraints:

- *`simd-complex`*`<basic_simd>` is `true`.
- `basic_simd<typename T::value_type, CAbi>::size() == basic_simd::size()` is `true`.

Effects:

- Replaces each element of the `basic_simd` object such that the i^{th} element is replaced with `operator[](i).real(v[i])` or `operator[](i).imag(v[i])`, respectively, for all `i` in the range of `[0, size())`.

§ 6.8. Add new section [**simd.complex_math**]

simd complex math [simd.complex_math]

Unless otherwise specified, each `complex` function declared in `<simd>` yields a `basic_simd` result, where the value of each element approximates the result obtained by applying the function with the same name from `<complex>` to the corresponding elements of the arguments. The value of an element of the result is unspecified if a domain, pole, or range error occurs for the corresponding elements of the arguments. `errno` is never modified.

```
template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> real(cor
template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> imag(cor

template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> abs(cons
template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> arg(cons
template<simd-complex V>
  constexpr rebind_simd_t<typename V::value_type::value_type, V> norm(cor
template<simd-complex V> constexpr V conj(const V&);
template<simd-complex V> constexpr V proj(const V&);

template<simd-complex V> constexpr V exp(const V& v);
template<simd-complex V> constexpr V log(const V& v);
template<simd-complex V> constexpr V log10(const V& v);

template<simd-complex V> constexpr V sqrt(const V& v);
template<simd-complex V> constexpr V sin(const V& v);
template<simd-complex V> constexpr V asin(const V& v);
template<simd-complex V> constexpr V cos(const V& v);
template<simd-complex V> constexpr V acos(const V& v);
template<simd-complex V> constexpr V tan(const V& v);
template<simd-complex V> constexpr V atan(const V& v);
template<simd-complex V> constexpr V sinh(const V& v);
template<simd-complex V> constexpr V asinh(const V& v);
template<simd-complex V> constexpr V cosh(const V& v);
template<simd-complex V> constexpr V acosh(const V& v);
template<simd-complex V> constexpr V tanh(const V& v);
template<simd-complex V> constexpr V atanh(const V& v);
```

Returns:

A data-parallel object initialized with values equal to the results of applying *cmplx-func* element-wise to `x`, where *cmplx-func* is the corresponding scalar function from `<complex>`.

```
template<simd-floating-point V>
  rebind_simd_t<complex<typename V::value_type>, V> polar(const V& x, cor

template<simd-complex V> constexpr V pow(const V& x, const V& y);
```

Returns:

A data-parallel object initialized with values equal to the results of applying the *cmplx-func* element-wise to `x` and `y`, where *cmplx-func* is the corresponding scalar binary function from `<complex>`.

7. Polls

7.1. Kona 2023 LEWG polls

Poll: Forward P2663R4 (Interleaved complex support in std::simd) with modified wording for 5.1 to constrain `complex<>` to floating point to LWG for C++26 (to be confirmed with a Library Evolution electronic poll).

SF	F	N	A	SA
8	**8**	**0**	**0**	**0**

7.2. Kona 2022 SG1 polls

Poll: After significant experience with the TS, we recommend that the next version (the TS version with improvements) of `std::simd` target the IS (C++26)

SF	F	N	A	SA
10	**8**	**0**	**0**	**0**

Poll: Future papers and future revisions of existing papers that target `std::simd` should go directly to LEWG. (We do not believe there are SG1 issues with `std::simd` today.)

SF	F	N	A	SA
9	8	0	0	0

8. Revision History

R5 => R6

- Updated wording section to reflect the changes and style now used in [P1928R13].
- Addressed LWG review comments
- Raised some LEWG design issues that need to be reconfirmed.

R4 => R5

- Modified wording to clarify that only floating-point specializations of `std::complex<>` should be allowed.
- Removed design option for making `real`/`imag` setter free-functions. The member function API from `std::complex` will be used.

R3 => R4

- Updated wording to reflect changes made to `std::simd` in [P1928R6] (e.g., removal of `fixed_size_simd`, addition of `basic_simd`).

R2 => R3

- Added wording

R1 => R2

- Add more content for proposal overview
- Rewrite to bikeshed

R0 => R1

- Add polls from Kona SG1 2022

§ References

§ Informative References

[P1928R6]

Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 19 June 2023. URL: https://wg21.link/p1928r6

[P2638R0]

Daniel Towner. *Intel's response to P1915R0 for std::simd parallelism in TS 2*. 22 September 2022. URL: https://wg21.link/p2638r0